

```
"""
```

SNS 광고 소비심리 자극 측정 모델

- 가중치 계산식
- KoBERT 파인튜닝용 데이터셋 구성
- 점수 산출 파이프라인

```
"""
```

```
import numpy as np
import pandas as pd
from dataclasses import dataclass
from typing import List, Dict
from sklearn.model_selection import train_test_split
from transformers import (
    AutoTokenizer,
    AutoModelForSequenceClassification,
    TrainingArguments,
    Trainer,
)
import torch
import torch.nn.functional as F

# =====
# 1. 가중치 설정
# 선행 메타분석 논문에서 수집한 r값을 정규화하여 가중치 산출
# 공식:  $w_i = r_i / \sum r$ 
# =====

# 각 카테고리의 피어슨 상관관계수(r값) - 선행 메타분석 기반
R_VALUES = {
    "권위_신뢰": 0.44, # Pornpitakpan (2004)
    "사회적_증명": 0.317, # Qiu & Zhang (2024)
    "사회적_정체성": 0.31, # Aguirre-Rodriguez et al. (2012)
    "호혜성": 0.30, # Cialdini(1984) + Raghurir(2004) 추정
    "희소성": 0.25, # Ali et al. (2025)  $\beta \rightarrow r$  변환
    "긴급성": 0.19, # Ladeira et al. (2023)
    "가격비교": 0.18, # 이상수 (2023)  $\beta \rightarrow r$  변환
}

# 전체 r값 합계 (정규화 분모)
SIGMA_R = sum(R_VALUES.values()) # 1.987
```

```

# 정규화 가중치: 각 r값을 합계로 나눠서 비율(%)로 변환
WEIGHTS = {cat: r / SIGMA_R for cat, r in R_VALUES.items()}

# 카테고리 → 숫자 라벨 매핑 (KoBERT 학습용)
# 0: 해당없음, 1~7: 각 카테고리
LABEL2IDX = {
    "해당없음": 0,
    "권위_신뢰": 1,
    "사회적_증명": 2,
    "사회적_정체성": 3,
    "호혜성": 4,
    "희소성": 5,
    "긴급성": 6,
    "가격비교": 7,
}

# 숫자 라벨 → 카테고리 이름 (추론 시 역변환용)
IDX2LABEL = {v: k for k, v in LABEL2IDX.items()}

# 전체 라벨 수 (KoBERT 출력층 크기 설정용)
NUM_LABELS = len(LABEL2IDX) # 8

# =====
# 2. 점수 계산
# KoBERT 분류 결과(카테고리 + 신뢰도)를 받아서 최종 점수 산출
# 공식: Score =  $\sum(w_i \times \text{confidence}_i) / n \times 100$ 
# =====

@dataclass
class SentenceResult:
    """문장 하나의 KoBERT 분류 결과를 담는 데이터 클래스"""
    sentence: str # 원본 문장
    category: str # KoBERT가 분류한 카테고리
    confidence: float # 분류 신뢰도 (softmax 확률, 0~1)
    weight: float # 해당 카테고리의 가중치
    score: float # 이 문장의 기여 점수 (weight × confidence)

def calculate_ad_score(sentence_results: List[SentenceResult]) -> dict:
    """

```

광고 전체의 소비심리 자극 점수를 계산하는 함수

Args:

sentence_results: 각 문장의 KoBERT 분류 결과 리스트

Returns:

final_score: 최종 점수 (0~100점)

raw_score: 정규화 전 원점수

n_sentences: 전체 문장 수

category_scores: 카테고리별 기여 점수

category_counts: 카테고리별 탐지 횟수

"""

카테고리별 누적 점수 초기화

category_scores = {cat: 0.0 for cat in WEIGHTS}

category_counts = {cat: 0 for cat in WEIGHTS}

각 문장의 점수를 카테고리별로 누적

for result in sentence_results:

if result.category in WEIGHTS:

가중치 × 신뢰도 = 이 문장의 기여 점수

category_scores[result.category] += WEIGHTS[result.category] *
result.confidence

category_counts[result.category] += 1

전체 카테고리 점수 합산

raw_score = sum(category_scores.values())

문장 수로 나눠서 정규화 후 100점 스케일 변환

n = len(sentence_results)

final_score = (raw_score / n * 100) if n > 0 else 0

return {

"final_score": round(final_score, 4),

"raw_score": round(raw_score, 4),

"n_sentences": n,

"category_scores": {k: round(v, 4) for k, v in category_scores.items()},

"category_counts": category_counts,

}

=====

```
# 3. 라벨링 데이터 → KoBERT 학습용 포맷 변환
# 엑셀 라벨링 파일을 KoBERT가 읽을 수 있는 형태로 변환
# =====
```

```
def normalize_cols(df):
    """컬럼명 정규화 (공백, 특수문자 → 언더스코어)"""
    return df.rename(columns=lambda c: c.strip()
                      .replace(" ", "_").replace(".", "_").replace("-", "_"))
```

```
def prepare_kobert_dataset(df: pd.DataFrame) -> pd.DataFrame:
    """
```

라벨링 완료 데이터프레임을 KoBERT 학습용으로 변환

처리 방식:

- 카테고리 없음 → label = 0 (해당없음)
- 카테고리 1개 → 해당 카테고리 라벨
- 카테고리 복수 → 가중치 가장 높은 카테고리 우선 채택

Args:

df: 라벨링 완료 데이터프레임

Returns:

text, label, label_name 컬럼을 가진 학습용 데이터프레임

"""

데이터프레임에 실제로 존재하는 카테고리 컬럼만 추출

```
cat_cols = [c for c in WEIGHTS.keys() if c in df.columns]
```

```
records = []
```

```
for _, row in df.iterrows():
```

```
    text = str(row["text"]).strip()
```

```
    if not text or text == "nan":
```

```
        continue # 빈 텍스트 스킵
```

```
    # 해당 행에서 값이 1인 카테고리 추출
```

```
    matched = [
```

```
        cat for cat in cat_cols
```

```
        if pd.to_numeric(row.get(cat, 0), errors="coerce") == 1
```

```
    ]
```

```
    if not matched:
```

```

        label_name = "해당없음"          # 아무 카테고리도 없으면 0
    elif len(matched) == 1:
        label_name = matched[0]          # 카테고리 1개면 그대로
    else:
        # 복수 카테고리면 가중치 높은 것 우선 채택
        label_name = max(matched, key=lambda x: WEIGHTS.get(x, 0))

    records.append({
        "text":      text,
        "label":      LABEL2IDX[label_name], # 숫자 라벨
        "label_name": label_name,            # 카테고리 이름
    })

return pd.DataFrame(records)

# =====
# 4. KoBERT 파인튜닝
# 사전학습된 한국어 BERT 모델을 우리 데이터로 추가 학습
# =====

# 클래스 가중치: 데이터 불균형 보정 + 중요 카테고리 강조
# 가중치 높은 카테고리를 틀리면 벌점을 더 크게 부과
CLASS_WEIGHTS = torch.tensor([
    0.1,                # 해당없음 (낮은 가중치)
    WEIGHTS["권위_신뢰"], # 22.1%
    WEIGHTS["사회적_증명"], # 15.9%
    WEIGHTS["사회적_정체성"], # 15.6%
    WEIGHTS["호혜성"], # 15.1%
    WEIGHTS["희소성"], # 12.6%
    WEIGHTS["긴급성"], # 9.6%
    WEIGHTS["가격비교"], # 9.1%
], dtype=torch.float)

class WeightedTrainer(Trainer):
    """클래스 가중치를 적용한 커스텀 학습기"""

    def compute_loss(self, model, inputs, return_outputs=False, **kwargs):
        labels = inputs.pop("labels")
        outputs = model(**inputs)

```

```

# 가중치 적용 손실 함수: 중요 카테고리 오분류 시 벌점 증가
loss = torch.nn.CrossEntropyLoss(
    weight=CLASS_WEIGHTS.to(outputs.logits.device)
)(outputs.logits, labels)

return (loss, outputs) if return_outputs else loss

```

```
def train_kobert(train_df, val_df):
```

```
    """
```

```
    KoBERT 파인튜닝 실행 함수
```

```
    Args:
```

```
        train_df: 학습 데이터 (80%)
```

```
        val_df:   검증 데이터 (20%)
```

```
    Returns:
```

```
        학습 완료된 model, tokenizer
```

```
    """
```

```
    # 한국어 BERT 모델 로드 (klue/bert-base 또는 snunlp/KR-ELECTRA)
```

```
    MODEL_NAME = "klue/bert-base"
```

```
    tokenizer = AutoTokenizer.from_pretrained(MODEL_NAME)
```

```
    model = AutoModelForSequenceClassification.from_pretrained(
```

```
        MODEL_NAME,
```

```
        num_labels=NUM_LABELS # 출력층 크기 = 8 (카테고리 수)
```

```
    )
```

```
def tokenize(batch):
```

```
    """텍스트를 BERT 입력 형식(토큰)으로 변환"""
```

```
    return tokenizer(
```

```
        batch["text"],
```

```
        padding="max_length", # 짧은 문장은 패딩으로 채움
```

```
        truncation=True,      # 128토큰 초과 시 자름
```

```
        max_length=128,
```

```
    )
```

```
from datasets import Dataset
```

```
# 판다스 데이터프레임 → HuggingFace Dataset 형식으로 변환 후 토큰나이징
```

```
train_ds = Dataset.from_pandas(train_df).map(tokenize, batched=True)
```

```
val_ds   = Dataset.from_pandas(val_df).map(tokenize, batched=True)
```

```

# 학습 하이퍼파라미터 설정
training_args = TrainingArguments(
    output_dir="/kobert_ad_classifier", # 모델 저장 경로
    num_train_epochs=5,                 # 전체 데이터 5번 반복
    per_device_train_batch_size=16,     # 한 번에 16개 문장 처리
    per_device_eval_batch_size=32,
    learning_rate=2e-5,                 # 학습률
    warmup_ratio=0.1,                   # 초반 10% 구간 학습률 서서히 증가
    eval_strategy="epoch",               # 매 에폭마다 검증
    save_strategy="epoch",               # 매 에폭마다 저장
    load_best_model_at_end=True,         # 학습 끝나면 가장 좋은 모델 로드
    metric_for_best_model="f1",          # F1 점수 기준으로 최적 모델 선택
)

```

```

def compute_metrics(eval_pred):
    """검증 시 정확도와 F1 점수 계산"""
    from sklearn.metrics import f1_score, accuracy_score
    preds = np.argmax(eval_pred.predictions, axis=-1)
    return {
        "accuracy": accuracy_score(eval_pred.label_ids, preds),
        "f1":       f1_score(eval_pred.label_ids, preds, average="weighted"),
    }

```

```

# 학습 실행
trainer = WeightedTrainer(
    model=model,
    args=training_args,
    train_dataset=train_ds,
    eval_dataset=val_ds,
    compute_metrics=compute_metrics,
)
trainer.train()

```

```

return model, tokenizer

```

```

# =====
# 5. 추론 파이프라인
# 학습 완료된 모델로 실제 광고 텍스트 분석
# =====

```

```
def predict_and_score(ad_text: str, model, tokenizer) -> dict:
    """
```

광고 텍스트 입력 → 소비심리 자극 점수 출력

Pipeline:

1. 텍스트를 문장 단위로 분리
2. KoBERT로 각 문장 카테고리 분류 + 신뢰도 출력
3. 가중치 × 신뢰도 = 문장별 기여 점수
4. 전체 점수 정규화 후 출력

Args:

ad_text: 분석할 광고 텍스트
model: 파인튜닝 완료된 KoBERT 모델
tokenizer: KoBERT 토큰나이저

Returns:

최종 점수 및 카테고리별 상세 결과

"""

```
import re
```

```
# 마침표, 느낌표, 물음표, 줄바꿈 기준으로 문장 분리
```

```
sentences = [s.strip() for s in re.split(r"[.!?|\n]", ad_text) if s.strip()]
```

```
sentence_results = []
```

```
for sentence in sentences:
```

```
    # 문장을 토큰으로 변환
```

```
    inputs = tokenizer(
```

```
        sentence,
```

```
        return_tensors="pt",
```

```
        padding=True,
```

```
        truncation=True,
```

```
        max_length=128,
```

```
    )
```

```
# 모델 추론 (gradient 계산 불필요하므로 no_grad)
```

```
with torch.no_grad():
```

```
    probs = F.softmax(model(**inputs).logits, dim=-1)[0]
```

```
# 가장 높은 확률의 카테고리 선택
```

```
pred_idx = probs.argmax().item()
```



```

confidence = probs[pred_idx].item() # 해당 카테고리 신뢰도
category    = IDX2LABEL[pred_idx]    # 숫자 → 카테고리 이름 변환

sentence_results.append(SentenceResult(
    sentence=sentence,
    category=category,
    confidence=confidence,
    weight=WEIGHTS.get(category, 0),
    score=WEIGHTS.get(category, 0) * confidence,
))

return calculate_ad_score(sentence_results)

# =====
# 6. 실행
# =====
if __name__ == "__main__":

    # 라벨링 파일 4개 불러오기
    df_sy = normalize_cols(pd.read_excel("이서영_직접체크용.xlsx", sheet_name="라벨링
체크"))
    df_dh = normalize_cols(pd.read_excel("김도훈.xlsx", sheet_name="라벨링"))
    df_mj = normalize_cols(pd.read_excel("김민정.xlsx"))
    df_sm = normalize_cols(pd.read_csv("안설민_라벨링_완.csv", skiprows=2))

    # 4개 파일 하나로 합치기
    df_all = pd.concat([df_sy, df_dh, df_mj, df_sm], ignore_index=True)

    # KoBERT 학습용 포맷으로 변환
    kobert_df = prepare_kobert_dataset(df_all)

    # 라벨 분포 확인
    print("라벨 분포:")
    print(kobert_df["label_name"].value_counts())

    # 학습(80%) / 검증(20%) 분리
    # stratify: 각 카테고리 비율 유지하며 분리
    train_df, val_df = train_test_split(
        kobert_df, test_size=0.2, random_state=42,
        stratify=kobert_df["label"]

```

```

)
print(f"\n학습: {len(train_df)}개 / 검증: {len(val_df)}개")

# KoBERT 파인튜닝 실행
model, tokenizer = train_kobert(train_df, val_df)

# 예시 광고 점수 측정
result = predict_and_score(
    "피부과 전문의 추천! 선택순 100명 한정. 오늘까지만 50% 할인.",
    model, tokenizer
)
print(f"\n최종 점수: {result['final_score']}점 / 100점")
print("\n카테고리별 기여 점수:")
for cat, score in sorted(result["category_scores"].items(), key=lambda x: -x[1]):
    if score > 0:
        print(f" {cat}: {score:.4f}")

//파이썬임

```